REACT + TYPESCRIPT DAY 2

Typing useRef

- Two "flavors" of useRef: DOM ref vs mutable value ref, and how their types differ.
- useRef<HTMLDivElement>(null) — why the generic + null initial value combo.
- Read-only vs mutable refs: RefObject<T> vs MutableRefObject<T>.
- Why .current is nullable and how TS forces you to null-check before use.
- Using refs to store timers/intervals (useRef<ReturnType<typeof setTimeout>>).
- forwardRef with TypeScript — typing both the ref and the props.

Typing useEffect

- Why useEffect itself rarely needs explicit generics — it's about typing what's inside.
- Typing cleanup functions (must return void or a cleanup function, not anything else).
- Dependency array pitfalls TS can catch (stale closures, missing deps).
- Typing async logic inside useEffect (why you can't make the callback itself async).
- Common gotcha: effect callback return type errors.

Typing useContext

- Creating a typed Context: createContext<ContextType | undefined>(undefined).
- Why undefined as default + a custom hook + runtime check is the standard safe pattern.
- Writing a useMyContext() custom hook that throws if used outside a Provider (and how that removes the need for | undefined everywhere else).
- Typing the Provider's value prop and children.

Typing useReducer

- Typing state shape and action shape separately.
- Using a discriminated union for actions (type: 'increment' | 'decrement' | 'set').
- Writing a typed reducer function: (state: State, action: Action) => State.
- How TS narrows the action payload based on action.type inside the reducer (switch statement narrowing).
- When to reach for useReducer vs useState from a typing-complexity perspective.

Custom Hooks

- Typing custom hook parameters like normal function params.
- Typing custom hook return values — object vs array return (and why array returns need as const).
- Generic custom hooks (e.g. useLocalStorage<T>(key: string, initial: T)).
- Reusing types/interfaces across multiple hooks (shared types file).

Typing API Calls / Async Data

- Typing fetch responses — why the response body is any by default and how to assert/validate it.
- Defining response interfaces/types that mirror your API shape.
- Typing loading/error/data state (a mini discriminated union: idle | loading | success | error).
- Brief mention of runtime validation (zod/io-ts) vs just trusting a type assertion.
- Typing async functions: Promise<T> return types.

Generic Components (deeper dive)

- Writing a generic <List<T>> or <Table<T>> component with a typed items: T[] and renderItem: (item: T) => ReactNode.
- Constraining generics with extends (e.g. T extends { id: string }).
- Generic components + forwardRef together (the tricky syntax workaround).

Utility Types Useful in React

- Partial<T>, Pick<T>, Omit<T> for deriving prop types from existing interfaces.
- ComponentProps<typeof SomeComponent> to reuse another component's prop types.
- ReturnType<typeof someHook> to infer a hook's return type.

Wrap-up

Mini exercise: build a typed useFetch<T>(url: string) hook combining generics, useState, useEffect, and discriminated union status state.